



Institut Teknologi Bandung

Directorate of Sustainable Professional Education

CHAPTER 6

The Universal Workflow of Machine Learning

You do not start from a dataset. You start from a problem — and the hardest work happens before the first line of model code.

Duration 3 hours (2 sessions)

Instructor Prof. Bambang Riyanto Trilaksono · Rahman Indra Kesuma, S.Kom., M.Cs.

Source Chollet & Watson, Deep Learning with Python 3e — chapter 6

Session Objectives

By the end of this session, participants can:

- 1 Frame a business problem as the right **kind of machine learning task** — including recognising when machine learning is **not** the answer.
- 2 Name the **two hypotheses** you make silently at the start of every project, and what it means if either is false.
- 3 Design collection and annotation so the data **represents production**, and recognise **sampling bias, target leaks**, and **concept drift**.
- 4 Choose a success measure, then choose the **last-layer activation, loss, and metrics** that match it.
- 5 Run the three development stages: **beat a baseline** → **scale up until it overfits** → **regularise and tune**.
- 6 Choose a deployment route — **REST API, on device, or in the browser** — and optimise the model for inference.
- 7 Set stakeholder expectations in **false-positive and false-negative** terms rather than "98% accurate".

BOOK

[Chapter 6 — full text](#)

NOTEBOOK

[01_understand_data_before_modelling.ipynb](#)

NOTEBOOK

[02_common_preprocessing.ipynb](#)

NOTEBOOK

[03_export_and_quantize.ipynb](#)

NOTEBOOK

[All chapter 6 notebooks](#)

In the real world, `keras.datasets` does not exist

Imagine opening your own machine learning consultancy. The projects start rolling in — and **not one of them arrives with its dataset**

Photo search

Type *wedding*, get every wedding photo — with no manual tagging.

Spam and abuse

Flagging posts in a growing chat app.

Music recommendation

For the users of an online radio station.

Credit-card fraud

For an e-commerce site.

Ad click-through rate

Deciding which ad to serve to whom, and when.

Defective biscuits

Flagging anomalies on a factory conveyor belt.

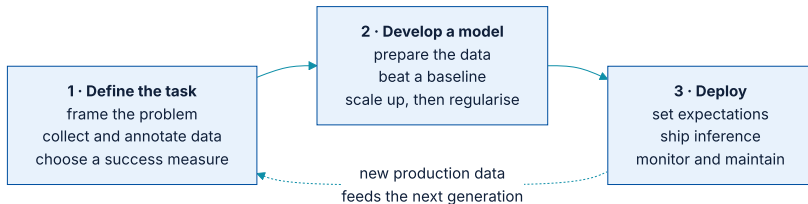
Archaeological sites

Locating undiscovered ones from satellite imagery.

...and your own case

Which you will carry through this chapter as the group assignment.

Three parts — and the hard one comes first



Note the return arrow: the loop never actually closes.

Model development is only one step in the machine learning workflow, and if you ask us, it's not the most difficult one. The hardest things in machine learning are framing problems and collecting, annotating, and cleaning data.

Chollet & Watson, section 6.2

01

Defining the task

You cannot do good work without deeply understanding the context.

Four questions to keep at the front of your mind

- 1 **What will the input data be? What are you trying to predict?** You can only learn to predict something you have training data for. **Data availability is usually the limiting factor here.**
- 2 **What type of task is this?** Binary? Multiclass? Scalar regression? Segmentation? Ranking? Or — quite possibly — **machine learning is not the best way** and plain statistics would serve better.
- 3 **What do existing solutions look like?** Perhaps a hand-crafted algorithm full of nested `if` statements. Perhaps a person doing it manually today. Understand what is already in place.
- 4 **Are there particular constraints?** End-to-end encryption may force the model onto the user's phone. Latency may force it onto an embedded device at the factory rather than a remote server.

Why the third question matters more than it looks

In most real projects the baseline is **not** "random". It is a rules-based system that has been running for years and that people already trust — and **that is a much harder thing to beat**

It is also the thing you will be compared against when the project is reviewed, so it should be measured properly at the start rather than described from memory.

The seven examples, mapped to task types

PROJECT	TASK TYPE	NOTE
Photo search	Multiclass, multilabel	—
Spam	Binary	Becomes three-way if <i>offensive content</i> is a separate class.
Music recommendation	Not deep learning	Better handled by matrix factorisation (collaborative filtering).
Credit-card fraud	Binary	—
Click-through rate	Scalar regression	—
Defective biscuits	Binary	But needs object detection first, to crop the biscuits out of the raw images. The book notes that <i>anomaly detection</i> would not fit here
Archaeological sites	Image similarity ranking	Retrieve images most like known sites.

Two lessons hiding in that table

Some problems are not deep learning problems

The music recommender is better served by collaborative filtering. Recognising that early saves months.

Some need two models in sequence

The biscuit case needs detection before classification. Framing it as one model would fail quietly.

Both are missed routinely when a team jumps straight to modelling.

The two hypotheses you are making silently

Your targets can be predicted from your inputs

That the relationship exists at all.

Your data is sufficiently informative

That it is enough to learn that relationship.

Until you have a working model these are **merely hypotheses**. Assembling examples of X and targets Y **does not mean X contains enough information to predict Y**

The book's example: predicting a stock's movement from its recent price history is **unlikely to succeed**, because price history does not contain much predictive information.

Technology is never neutral

- 1 **The validity is already in doubt** — it is not clear why trustworthiness would be reflected in a face.
- 2 Collecting a dataset would amount to **recording the biases and prejudices of the people labelling the pictures**.
- 3 A model trained on it would merely **encode those same biases into a black-box algorithm** — which gives them a thin veneer of legitimacy.

Why the veneer is the dangerous part

In a largely tech-illiterate society like ours, "The AI algorithm said this person cannot be trusted" strangely appears to carry more weight and objectivity than "John Smith said this person cannot be trusted" — despite the former being a learned approximation of the latter.

Chollet & Watson, section 6.1.1

Technical choices are also ethical choices. If your work has any impact on the world, that impact has a moral direction — so **be deliberate about the values your work supports**

Collecting data is the expensive part — and the highest-return one

If you get an extra 50 hours to spend on a project, chances are that the most effective way to allocate them is to collect more data, rather than search for incremental modeling improvements.

Chollet & Watson, section 6.1.2

The point that data matters more than algorithms was made most famously in a 2009 Google paper, "*The Unreasonable Effectiveness of Data*" — itself a play on Eugene Wigner's 1960 title. That predates the popularity of deep learning, and the rise of deep learning has **only increased the importance of data**

Who should do the annotating

OPTION	UPSIDE	RISK
Annotate it yourself	Full control over quality.	Slow and costly in time.
Crowdsourcing (e.g. Mechanical Turk)	Inexpensive and scales well.	Annotations may end up quite noisy
A specialist labelling company	Saves time and money.	Takes away control.

Your annotation process determines the quality of your targets, which in turn determines the quality of your model. It deserves the same scrutiny as the architecture.

Three questions that decide which option

- ♦ **Do the labellers need to be subject-matter experts?** Cat versus dog can be done by anyone; dog breeds need specialised knowledge; annotating CT scans of fractures **pretty much requires a medical degree**.
- ♦ **If expertise is needed, can you train people?** If not, how will you get access to the relevant experts at all?
- ♦ **Do you yourself understand how the experts arrive at the annotations?** If not, your dataset becomes a black box and manual feature engineering is closed to you — not fatal, but limiting.

And decide early what software will record the annotations. Productive annotation tooling **saves a great deal of time**, so it is worth investing in **at the start of a project, not the middle**

Non-representative data — the cardinal sin

After release: **wrong 8 times out of 10**. User photos — random dishes, random restaurants, random phones — **look nothing like** the professional, well-lit, appetising pictures it was trained on.

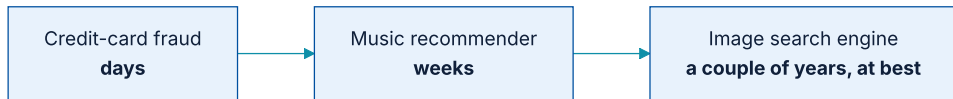
- ♦ Where possible, collect data **directly from the environment the model will be used in**.
- ♦ A review sentiment model belongs on new IMDB reviews — not Yelp reviews, not tweets.
- ♦ If training on production data is impossible, **understand exactly how they differ** and correct for it actively.

Sampling bias: DEWEY DEFEATS TRUMAN

On election night in 1948 the *Chicago Tribune* printed the headline "**DEWEY DEFEATS TRUMAN**". By morning Truman had won. The editor had trusted a telephone survey — but telephone users in 1948 **were not a representative sample of voters**. They were likelier to be richer, more conservative, and to vote for Dewey.

Every phone survey now accounts for this. That does not make sampling bias a thing of the past — **only a thing pollsters are now aware of**

Concept drift, and how long a model lives



The book's own figures. The most adversarial problem is the one that goes stale fastest.

Concept drift is when the properties of production data change over time, gradually degrading the model. The IMDB dataset was collected in 2011; a model trained on it does worse on 2020 reviews than on 2012 ones, because vocabulary, expressions, and genres move.

The assumption underneath every deployed model

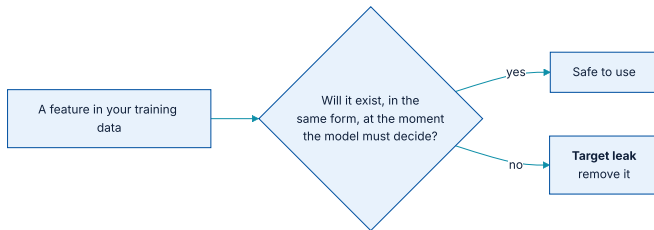
Machine learning can only memorise patterns **present in the training data**. Using a model trained on the past to predict the future assumes **the future will behave like the past**. **Often it does not**.

Handling fast drift requires continuous data collection, annotation, and retraining — which is a staffing decision as much as a technical one.

Understand your data before you model it

- 1 Images or text? **Look at a few samples directly**, with their labels.
- 2 Numeric features? **Plot the histograms** to feel the ranges and frequencies.
- 3 Location data? **Put it on a map**. Do patterns appear?
- 4 Any **missing values**? They must be handled during preparation.
- 5 A classification task? **Print the count per class**. If it is not balanced, that has to be accounted for.
- 6 Check for **target leaks**.

Target leaking, and the one question that catches it



Ask it of every column, not just the suspicious ones.

The book's example: training on medical records to predict whether someone will be treated for cancer, where the records include the feature *"this person has been diagnosed with cancer"*.

The common real-world shape: **fields filled in later by an operator** — handling status, closure code, follow-up notes. None of them exist **at the moment the model has to decide**

Choosing a measure of success

To control something, you need to be able to observe it. To achieve success on a project, you must first define what you mean by success.

Chollet & Watson, section 6.1.4

PROBLEM SHAPE

Balanced classification

Imbalanced, ranking, multilabel

Neither of the above

COMMON METRICS

Accuracy, and **AUC** of the ROC curve.

Precision and recall, or a metric counting false/true positives and negatives.

It is not unusual to have to **define your own metric**.

The metric guides **every technical decision** in the project, so it must align directly with the higher-level goal — **the customer's business success**

02

Developing a model

The part every tutorial covers — and not the hardest one.

Vectorisation and normalisation

the two properties you want

PYTHON

```
# 1. Take small values - most values in the 0-1 range
# 2. Be homogeneous - all features on roughly the same scale

# The stricter, common practice:
x -= x.mean(axis=0) # each feature centred on 0
x /= x.std(axis=0) # each feature with unit standard deviation
```

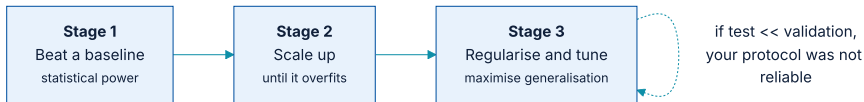
Feeding in large values (multi-digit integers) or heterogeneous ones (one feature 0–1, another 100–200) triggers **large gradient updates that prevent the network from converging.**

Missing values: categorical and numerical differ

MISSING VALUE IN A...	WHAT TO DO
Categorical feature	Safe to create a new category meaning "missing" . The model will learn what that implies about the target on its own.
Numerical feature	Avoid an arbitrary value such as 0 — it can create a discontinuity in the latent space. Use the mean or median , or train a model to predict the value from the other features.

A subtle trap: if you **expect** missing categorical values at test time but trained on complete data, the network **never learned to ignore them**. Fix it by generating training samples with holes — duplicate some rows and drop the fields you expect to lose.

Model development is three stages, in order



Doing them out of order — regularising before you can overfit — wastes the most time.

Stage 1 — beat a baseline

Feature engineering

Filter out uninformative features, and use knowledge of the problem to build likely-useful new ones.

A good enough training configuration

Which loss function? What batch size and learning rate?

The right architecture priors

Densely connected? ConvNet? Recurrent? Transformer? Or is deep learning even the right approach here?

The goal at this stage is **statistical power**: a small model that beats a simple baseline. Nothing more.

The reference table for that third choice

TASK	LAST-LAYER ACTIVATION	LOSS FUNCTION	METRICS
Binary classification	Sigmoid	Binary crossentropy	Binary accuracy, ROC AUC
Multiclass, single-label	Softmax	Categorical crossentropy	Categorical accuracy, top-k, ROC AUC
Multiclass, multi-label	Sigmoid	Binary crossentropy	Binary accuracy, ROC AUC
Regression	None	Mean squared error	Mean absolute error

Why you optimise a proxy instead of the metric you care about

ROC AUC satisfies neither, so it cannot be optimised directly. In classification we optimise a proxy — usually crossentropy — **hoping that as crossentropy falls, ROC AUC rises**

For most problems there are existing templates. You are not the first person to build a spam detector or an image classifier, so **research the prior art** before inventing.

Stage 2 — scale up until it overfits

- ♦ Add layers.
- ♦ Make the layers bigger.
- ♦ Train for more epochs.

The ideal model sits **exactly on the border** between underfitting and overfitting. **To find where the border is, you must cross it.**

Stage 3 — regularise and tune

- ♦ Try different architectures; add or remove layers.
- ♦ Add **dropout**.
- ♦ If the model is small, add **L1 or L2 regularisation**.
- ♦ Try different hyperparameters — units per layer, learning rate.
- ♦ Optionally iterate on **data curation or feature engineering** again.

Much of this can be automated with hyperparameter tuning software such as **KerasTuner** (chapter 18).

The leak that comes back at the end

Every time you use validation feedback to tune, information leaks into the model. A few times is harmless. Done **systematically over many iterations**, the model overfits **to the validation process itself** — even though no model was ever trained on validation data.

So if test performance turns out **significantly worse** than validation performance, either your validation procedure was not reliable, or you overfitted to it. The remedy is a more reliable protocol — **iterated K-fold**.

03

Deploying your model

A project does not end at a notebook that saves a trained model.

Do not say "98% accurate"

- ♦ Non-specialists often expect the system to **understand** its task and to exercise human-like common sense.
- ♦ The remedy: **show examples of how it fails** — especially misclassifications that look surprising.
- ♦ Avoid abstract statements like *"the model is 98% accurate"*, which **most people mentally round up to 100%**.

Speak in **false-negative and false-positive rates**, then translate them into daily volumes that a person can picture.

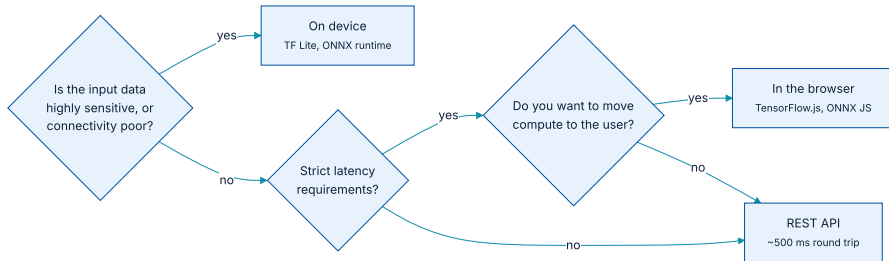
What that sounds like in practice

With these settings, the fraud detection model would have a 5% false-negative rate and a 2.5% false-positive rate. Every day, an average of 200 valid transactions would be flagged as fraudulent and sent for manual review, and an average of 14 fraudulent transactions would be missed. An average of 266 fraudulent transactions would be correctly caught.

The book's own worked example, section 6.3.1

Also discuss the **threshold** with stakeholders. Different thresholds give different error rates, and that trade-off **can only be resolved with deep knowledge of the business context**

Three deployment routes



The constraints choose the route, not the other way round.

Reading the tree: what each question is really asking

QUESTION	WHY IT COMES FIRST	WHERE A YES SENDS YOU
Sensitive data, or poor connectivity?	This one outranks the others because it is not a performance question. If the data may not leave the device, or there is no reliable link, no amount of latency budget changes the answer.	On device — TF Lite, ONNX runtime
Strict latency requirement?	A REST round trip costs roughly 500 ms before your model does anything. That is fine for a form submission and useless for anything reacting to a camera.	on to the next question
Move compute to the user?	Running in the browser costs you nothing per request and costs the user their battery. It also means shipping the model weights, which anyone can then keep.	In the browser — TensorFlow.js, ONNX JS

Every path that answers *no* ends at the REST API, and that is the right default: one copy of the model, one place to update it, one place to watch. Leave it only when a constraint above forces you to.

When each route is the right one

ROUTE	USE IT WHEN	EXAMPLE FROM THE BOOK
REST API	Reliable internet · no strict latency (round trip $\approx$ 500 ms) · input data is not highly sensitive, since it must be decrypted on the server.	Image search, recommender, fraud detection, satellite imagery.
On device	Strict latency or poor connectivity · the model can be made small enough · top accuracy is not mission-critical · input data must not be decryptable off-device.	Spam filter inside an end-to-end encrypted chat app; biscuit detection at the factory.
In the browser	You want to move compute to the user (server costs drop sharply) · data must stay on their machine · latency matters · it must work offline after download.	The web and desktop versions of the chat app.

One warning about the browser route

The **entire model is downloaded to the user's device**. Make sure nothing about it needs to stay confidential — because given a trained model it is usually possible to **recover some information about its training data**

Which means: do not publish a model that was trained on sensitive data, however convenient the deployment would be.

Exporting: TensorFlow Serving and ONNX

TensorFlow SavedModel

PYTHON

```
model.export("path/to/location",  
            format="tf_saved_model")  
  
reloaded = tf.saved_model.load("path/to/location")  
predictions = reloaded.serve(input_data)
```

ONNX

PYTHON

```
model.export("path/to/location",  
            format="onnx")  
  
ort_session = onnxruntime.InferenceSession(  
    "path/to/location")  
predictions = ort_session.run(None, input_data)
```

If that sounds like the compilation mechanism from chapter 3, it is: TensorFlow Serving is essentially **a library for serving `tf.function` graphs with a saved set of weights**

Where those exports run

- ♦ **TensorFlow Lite** — on-device inference for Android and iOS, ARM CPUs, Raspberry Pi, and some microcontrollers. Same save format as TF Serving. The ONNX runtime also runs on mobile.
- ♦ **TensorFlow.js** — runs in the browser and implements almost the whole Keras API (its working name was *WebKeras*). ONNX has its own JavaScript runtime.
- ♦ **Managed cloud services** such as Cloud AI Platform handle batching, load balancing, and scaling for you.

Optimising for inference



Both techniques trade a little accuracy for a lot of size and speed.

Pruning drops the coefficients that contribute least; how much you prune is your lever on the size-versus-accuracy trade-off. **Quantization** converts float32 weights to int8, giving a model **four times smaller** that stays near the original accuracy.

Quantization is one line

the built-in quantize API

PYTHON

```
model.quantize("int8")      # compress each weight down to a single byte
model.export("path/to/location", format="onnx")
```

Do this **before** importing into TensorFlow.js or exporting to TensorFlow Lite — it matters most where power and memory are tight, **phones and embedded devices**

Monitoring: pressing the button is not the end

- 1 **Randomised A/B testing.** Send a subset of cases through the new model and keep a control subset on the old process. Once enough cases have run, the difference **can be attributed to the model** rather than to everything else that changed.
- 2 **Regular manual audits** of predictions on production data. Reuse the annotation infrastructure: send a fraction out for labelling and compare.
- 3 **When manual audit is impossible**, use alternatives such as user surveys — the book's suggestion for the spam and abuse flagging system.

Maintenance starts on launch day

As soon as your model has launched, you should be getting ready to train the next generation that will replace it.

- ♦ Watch for **changes in production data**. Are new features available? Should the label set be expanded or edited?
- ♦ Keep collecting and annotating, and **keep improving the annotation pipeline** over time.
- ♦ Pay special attention to samples the current model finds **difficult** — those are **the ones most likely to improve performance**

What has to survive this chapter

- ❶ **Define the task first.** Understand the context, the goal, and the constraints; collect and annotate data; choose how success is measured.
- ❷ You always make **two hypotheses**. Until a model works, neither is proven.
- ❸ **Technical choices are ethical choices.** Technology is never neutral.
- ❹ **Data must represent production.** Watch for sampling bias, target leaks, and concept drift.
- ❺ **Beat a baseline** → **scale up until it overfits** → **regularise and tune**. In that order.
- ❻ **Deploy according to the constraints** — API, device, or browser — then prune and quantize for inference.
- ❼ **Talk in false positives and false negatives**, not percentages of accuracy.
- ❽ **No model lasts forever.** Monitor, audit, and prepare the successor from launch day.

NOTEBOOK

[03_export_and_quantize.ipynb](#)

NEXT

[Chapter 7 — A deep dive on Keras](#)